Ceph Performance Evaluation Report

Issei Hasegawa

2026-05-20

Table of contents

1 Overview

This study evaluates the performance of Ceph, a distributed storage system, by utilizing a block I/O trace. Ceph is designed to unify storage devices across various nodes, providing both durability and scalability. In particular, the RADOS Block Device (RBD) enables the creation of virtual block devices on top of Ceph, which can be accessed for reading and writing just like standard disks.

The purpose of this study is to replay a vSCSI block I/O trace named cloudPhysicsIO.vscsi on a Ceph cluster set up on CloudLab, while measuring and analyzing its performance throughout the replay process. Rather than relying on artificial benchmarks such as basic random reads or writes, this study employs a trace that more accurately reflects real-world storage access patterns. This approach allows for the observation of Ceph's behavior under a more authentic workload.

Regarding the performance evaluation, it primarily collects metrics such as throughput, IOPS, and latency. Using these metrics, we assess the performance of the Ceph cluster, identify any bottlenecks, and determine whether the observed results are reasonable given the workload and Ceph configuration.

This paper first describes the experimental environment and the configuration of the Ceph cluster, followed by an explanation of the characteristics of the vSCSI traces used and the measurement methods employed. Finally, based on the results obtained, we analyze the performance characteristics of Ceph and discuss the limitations of this experiment and areas for future improvement.

2 Experimental Environment

2.1 Hardware Environment

This study used three nodes on CloudLab's FAST25-AE project. The nodes used in this experiment, `node0`, `node1`, and `node2`, had the same hardware specifications. Each node is equipped with an Intel Xeon E5-2630 v3 CPU, 64 GB of DDR4 memory, an SSD for the operating system, and HDDs used as Ceph OSDs. Specifically, each node has two HDDs,

for a total of six HDDs used as Ceph OSDs. Additionally, a 10 GbE network was used for communication between nodes.

Component	Specification
Platform	CloudLab (FAST25-AE project)
Number of nodes	3 (`node0`, `node1`, `node2`)
CPU	Intel Xeon E5-2630 v3 @ 2.40 GHz, 8 cores
Memory	64 GB DDR4 2133 MHz ECC Registered
OS Disk	Intel SSDSC2BX20 200 GB SSD
OSD Disk	Seagate ST1000NM0033 HDD × 2 per node, 931 GB each
Network	Intel X710 10 GbE SFP+

With this configuration, Ceph stores data using HDDs distributed across multiple nodes. Consequently, the performance results of this experiment are influenced not only by the performance of individual disks but also by HDD access speeds, the inter-node network, and Ceph's replication processes.

2.2 Software Environment

This study used Ubuntu 22.04.2 LTS on each node, and installed Ceph 17.2.9 Quincy to set up the cluster. For Ceph storage, an RBD image was created and mapped as a block device, and the vSCSI trace was replayed against it. Regarding the trace replay, an originally created Python script was used, which reads block I/O called 'cloudPhysicsIO.vscsi' and issues actual read/write requests to '/dev/rbd0'. In addition, in order to analyze the cache performance, 'cachesim' from libCacheSim was utilized. The software environment is summarized in the following table:

Component	Version / Detail
Operating System	Ubuntu 22.04.2 LTS, Linux 5.15.0-168-generic
Ceph	17.2.9 Quincy
Trace replay tool	Custom Python script using synchronous writes with `O_SYNC` to `/dev/rbd0`
Cache simulation tool	libCacheSim `cachesim`
Trace file	`cloudPhysicsIO.vscsi`

This software environment enables a comparison between the I/O performance on an actual Ceph cluster and the behavior predicted by cache simulations.

3 Ceph Configuration

In this experiment, a Ceph cluster was set up using three nodes. This section describes the configuration of the Ceph cluster and the setting of storage used in this experiment.

3.1 Cluster Topology

The Ceph cluster in this experiment consists of three nodes, which are 'node0', 'node1', and 'node2'. Each node hosted a MON daemon, forming a quorum of three monitors. Additionally, an MGR daemon was configured with 'node0' as the active manager and 'node1' as the standby manager.

Six OSDs were set up in total. Each node had two HDDs and each HDD was used as a Ceph OSD. Specifically, 'osd.0' and 'osd.2' were placed on 'node0', 'osd.1' and 'osd.3' were placed on 'node1', and 'osd.4' and 'osd.5' were placed on 'node2'. BlueStore was used as the OSD backend. The Ceph cluster configuration is summarized below.

Component	Configuration
MON daemons	`node0`, `node1`, `node2`
MGR daemons	`node0` active, `node1` standby
OSD count	6 total
OSD placement	`node0`: `osd.0`, `osd.2`; `node1`: `osd.1`, `osd.3`; `node2`: `osd.4`, `osd.5`
OSD backend	BlueStore
Replication factor	3
Pool	`rbd` with 32 PGs
RBD image	`trace-disk`, 35 GB
Mapped device	`/dev/rbd0` on `node0`
BlueStore cache size	1 GB per OSD
Total raw capacity	5.5 TiB
Cluster health	`HEALTH_OK`

In this configuration, all three nodes participate in both cluster management and data storage.

3.2 Storage Configuration

This experiment utilized Ceph RBD to establish a block device. A 35 GB RBD image, designated as `trace-disk`, was created and mapped on `node0` as `/dev/rbd0`. The trace replay workload was performed on this mapped RBD device.

The replication factor was configured to 3. This indicates that each data segment is replicated across three OSDs. Such a configuration enhances durability, as the system can withstand certain OSD or node failures without data loss. However, it may also lead to increased write latency since each write operation must be replicated to multiple OSDs before receiving acknowledgment.

Each OSD employed BlueStore as its storage backend. BlueStore serves as the standard Ceph backend for the management of data blocks, metadata, and RocksDB-related information. In this experiment, the BlueStore cache size was allocated at 1 GB per OSD. Nevertheless, this cache is not exclusively reserved for user data; it is also utilized for metadata and RocksDB-related data. Consequently, the actual cache capacity available for user data may be less than the total 1 GB.

In summary, this storage configuration integrates three nodes, six OSDs, a replication factor of 3, and RBD to replay the trace within a realistic distributed storage environment.

4 Workload Analysis

In this experiment, we used a vSCSI block I/O trace named `cloudPhysicsIO.vscsi` as the workload. This trace records storage access patterns observed in actual cloud environments. As a result, it can reproduce I/O behavior in a Ceph cluster that is closer to real-world conditions than artificial benchmarks such as simple random read/write operations.

In this chapter, we first explain the format of the trace file, and then summarize the workload characteristics contained in the trace, such as the ratio of reads to writes, the number of requests, I/O size, and address space.

4.1 Trace Format

The trace file used, `cloudPhysicsIO.vscsi`, is a binary vSCSI trace. In this file, each I/O request is stored as a 32-byte record. Each record contains information such as the request number, I/O size, SCSI command, logical block number, and timestamp.

The record format of the trace is as follows.

Field	Size	Description
`sn`	4 bytes	Sequence number
`len`	4 bytes	I/O size in bytes
`nSG`	4 bytes	Scatter-gather count
`cmd`	2 bytes	SCSI command code
`ver`	2 bytes	Version field
`lbn`	8 bytes	Logical Block Number

Field	Size	Description
`ts`	8 bytes	Timestamp in microseconds

In this context, the `cmd` field was used to determine whether each request was a read or a write. Additionally, `lbn` indicates the logical block number of the target, and when actually issuing I/O to the Ceph RBD, this value was used as an offset in 512-byte units.

4.2 Workload Characteristics

The trace is write-dominant, with writes accounting for 58.7% of all 113,872 requests.

The main characteristics of the workload are as follows.

Metric	Value
Total requests	113,872
Read requests	46,974 (41.3%)
Write requests	66,898 (58.7%)
Total read data	1,714 MB
Total write data	2,297 MB
Trace duration	7,200 seconds
Average request rate	15.82 requests/sec
Most frequent I/O size	65,536 bytes
I/O size range	512 bytes to 69,632 bytes
Address space required	Approximately 31.3 GB

In this workload, the most frequently occurring I/O size was 65,536 bytes, or 64 KB. The I/O sizes ranged from 512 bytes to 69,632 bytes. The total duration of the trace was 7,200 seconds, or 2 hours, and the average request rate was approximately 15.82 requests per second.

Furthermore, this workload does not generate requests at regular intervals but exhibits burst-like behavior. While the number of requests is low during most time periods, there are certain periods where a large number of requests are concentrated within a short time frame. This behavior is considered to closely resemble I/O patterns in actual cloud storage environments.

Based on these characteristics, this trace represents a workload with a high write ratio, a skewed access pattern, and bursty behavior. By replaying such a workload on Ceph, we can evaluate how a distributed storage system behaves under realistic I/O patterns.

5 Measurement Method

This chapter describes the methods used to measure the performance of a Ceph cluster. In this experiment, we not only replayed traces against Ceph but also performed cache simulations using libCacheSim. This allowed us to compare the theoretically predicted cache behavior with the actual cache behavior observed in the Ceph BlueStore backend.

The measurements were conducted in three main steps. First, we used libCacheSim to simulate the cache miss rate for the traces using various cache algorithms and cache sizes. Next, we replayed the same traces as actual block I/O operations against a Ceph RBD device. Finally, we collected performance counters from Ceph BlueStore to analyze how many reads actually occurred on the BlueStore side and how many of those resulted in cache hits or cache misses.

5.1 Cache Simulation with libCacheSim

First, we used the `cachesim` tool from libCacheSim to simulate cache behavior for the `cloudPhysicsIO.vscsi` trace. The purpose of this simulation was to determine the likelihood of cache hits for this workload in an ideal cache environment.

In the simulation, we used four types of cache replacement algorithms: LRU, LFU, FIFO, and ARC. We also tested cache sizes of 10 MB, 50 MB, 100 MB, 500 MB, 1 GB, and 2 GB. This allowed us to observe how the miss rate changes with increasing cache size and to identify differences between the various algorithms.

This simulation assumes an ideal scenario where the cache is dedicated solely to data blocks and is not affected by other metadata or background processes. Therefore, the results from libCacheSim were used as a theoretical benchmark for comparison rather than as a direct representation of the actual cache performance of Ceph BlueStore.

5.2 Real I/O Replay on Ceph RBD

Next, we replayed the vSCSI trace as actual block I/O on a Ceph RBD device. To do this, we used a Python script that reads the trace file and interprets each record as a read or write request. The I/O was directed to the Ceph RBD device `/dev/rbd0`, which is mapped on `node0`.

For each request, we calculated the access offset based on the Logical Block Number (lbn) in the trace. Specifically, we multiplied the lbn by 512 bytes to determine the byte-level offset on the RBD device. We also used the `len` field in the trace for the I/O size. This allowed us to issue actual I/O operations that closely matched the access locations and sizes recorded in the trace.

For write requests, `O_SYNC` was used so that writes were completed synchronously rather than being acknowledged only after being buffered. However, this does not fully eliminate the influence of the Linux page cache, especially for read requests.

Additionally, the Linux page cache was flushed before starting the replay. This minimized the influence of the page cache at the start of the experiment. Requests were issued in a single-threaded manner, following the order of the traces. This was done to preserve the I/O order within the traces and to first observe the basic behavior of a single client.

5.3 BlueStore Statistics Collection

Finally, we used Ceph BlueStore's performance counters to measure the actual read and caching behavior occurring in the BlueStore backend. We used the `ceph tell osd.* perf dump` command to retrieve the performance counters for each OSD immediately before and after trace replay.

By calculating the difference between the counters before and after replay, we extracted the I/O activity generated by the trace replay. In particular, this experiment focused on counters related to BlueStore reads.

The main counters used are as follows.

Counter	Description
`read_random_count`	Total number of random reads received by BlueStore
`read_random_buffer_count`	Number of reads served from the BlueStore memory cache
`read_random_disk_count`	Number of reads that required disk access

Since read_random_buffer_count represents reads processed by BlueStore's memory cache, we treated it as a cache hit. On the other hand, since read_random_disk_count represents reads that actually required disk access, we treated it as a cache miss.

This approach allowed us to compare the cache miss rate predicted by libCacheSim with the cache miss rate observed in the actual BlueStore system.

6 Results

This chapter presents the results of cache simulations using libCacheSim, the results of real I/O replay on Ceph RBDs, and the cache performance of BlueStore. These results are used to compare the cache behavior predicted by the simulations with the performance observed on an actual Ceph cluster.

6.1 Cache Simulation Results

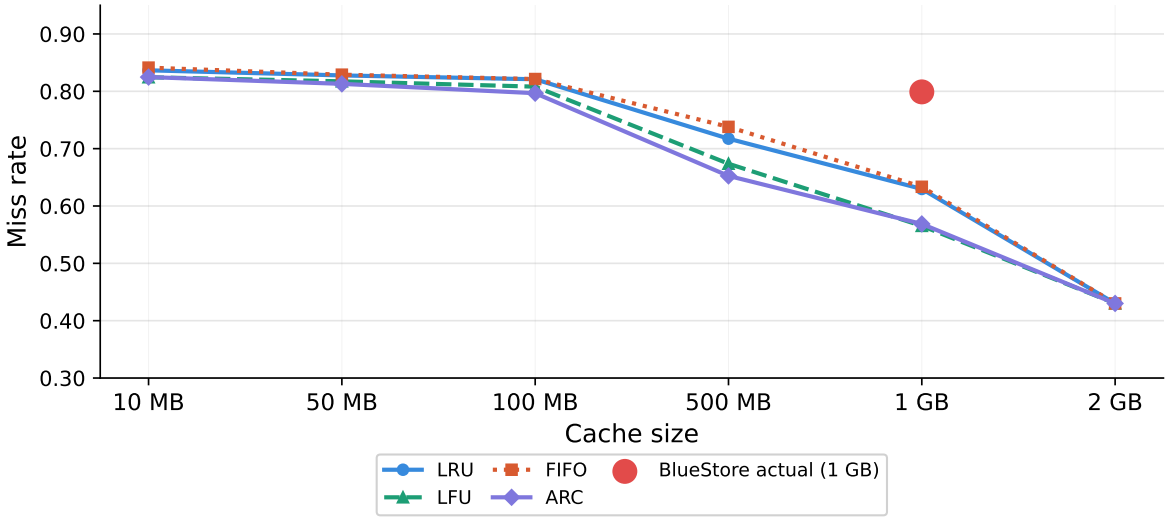

Figure 1: Cache miss rate vs. cache size for four eviction algorithms. The red point shows the BlueStore actual miss rate at 1 GB.

First, we used libCacheSim to perform cache simulations on the `cloudPhysicsIO.vscsi` trace. In the simulations, we used four different cache replacement algorithms—LRU, LFU, FIFO, and ARC—and measured the miss ratio for multiple cache sizes ranging from 10 MB to 2 GB.

Cache Size	LRU	LFU	FIFO	ARC
10 MB	0.8366	0.8245	0.8413	0.8248
50 MB	0.8277	0.8174	0.8290	0.8130
100 MB	0.8210	0.8081	0.8214	0.7967
500 MB	0.7174	0.6737	0.7380	0.6525
1 GB	0.6297	0.5653	0.6335	0.5687
2 GB	0.4301	0.4301	0.4301	0.4301

As a result, the miss ratio decreased for all algorithms as the cache size increased. In particular, with a cache size of 2 GB, all algorithms converged to the same miss ratio of 0.4301. This suggests that the primary working set size for this trace is approximately 2 GB.

Furthermore, with a 1 GB cache size, the LRU miss ratio was 0.6297, while the ARC miss ratio was 0.5687. This indicates that ARC performed relatively well for this workload because it takes both recency and frequency into account.

6.2 Real I/O Replay Results

Next, the same `cloudPhysicsIO.vscsi` trace was replayed as real block I/O against the Ceph RBD device `/dev/rbd0`. A total of 113,872 requests were replayed, and the experiment measured replay time, IOPS, throughput, and errors.

Metric	Value
Total requests	113,872
Total replay time	1,645 seconds
Overall IOPS	69.22
Read throughput	1.04 MB/s
Write throughput	1.40 MB/s
Total throughput	2.44 MB/s
Errors	0

The replay completed in 1,645 seconds with 69.22 IOPS, 2.44 MB/s total throughput, and no errors.

The latency results are shown below.

Latency Metric	All I/O	Read only	Write only
Average	14.442 ms	0.056 ms	24.543 ms
Median	16.687 ms	0.024 ms	22.342 ms
p95	33.256 ms	—	—
p99	78.505 ms	0.708 ms	95.260 ms
Max	227.357 ms	18.811 ms	227.357 ms

Write latency was significantly higher than the measured read latency, though the read latency figure is influenced by the Linux page cache as discussed in Section 7.3.

6.3 BlueStore Cache Performance

Finally, BlueStore performance counters were used to measure the cache behavior actually observed inside the Ceph backend. Since the BlueStore performance counters could not be reset between runs, the counters were recorded immediately before and after the replay, and the difference was used to isolate the I/O activity attributable to the trace replay.

BlueStore Metric	Value
Total random reads	1,902

BlueStore Metric	Value
Cache hits (`read_random_buffer_count`)	436
Cache misses (`read_random_disk_count`)	1,520
BlueStore cache hit rate	22.9%
BlueStore cache miss rate	79.9%

According to the BlueStore performance counters, 1,902 random reads reached BlueStore during the replay. Among them, 436 reads were served from the BlueStore memory cache, while 1,520 reads required disk access.

The observed BlueStore cache hit rate was 22.9%, and the cache miss rate was 79.9%. These values serve as the real-system measurements that can be compared with the idealized cache simulation results from libCacheSim.

Note that the sum of cache hits (436) and cache misses (1,520) equals 1,956, which slightly exceeds the total random read count of 1,902. This discrepancy likely arises because BlueStore's internal counters track different granularities of I/O operations, and some reads may be counted differently across the buffer and disk paths.

This section reports the results only. The reasons for the difference between the simulated cache behavior and the observed BlueStore cache behavior are discussed in the next section.

7 Discussion

This section discusses the meaning of the experimental results. In particular, it focuses on whether the observed write latency is reasonable for the Ceph configuration, why there is a gap between the cache simulation results and the actual BlueStore cache performance, and why the measured read latency should be interpreted carefully.

7.1 Write Latency

In this experiment, the average write latency was 24.543 ms, and the p99 write latency was 95.260 ms. These results are reasonable given the experimental environment.

The main reason is that a Ceph write operation is not a simple write to a single disk. In this experiment, the replication factor was set to 3. Therefore, each write request had to be replicated to three OSDs. From the client's perspective, a write can only complete after the required OSDs process the write and return acknowledgements.

In addition, this experiment used HDDs as OSD disks. HDDs are affected by seek time and rotational latency during random writes, so write latency is generally higher than it would be

with SSDs. Ceph's BlueStore commit process and network communication between nodes can also contribute to the final write latency.

7.2 Simulation vs. Real-World Gap

One of the central findings of this experiment is the gap between the cache miss ratio predicted by libCacheSim and the cache miss rate actually observed in BlueStore.

For a 1 GB cache size, the libCacheSim LRU simulation predicted a miss ratio of 62.97%. In contrast, the actual BlueStore performance counters showed a cache miss rate of 79.9%. This means that the observed BlueStore miss rate was about 16.9 percentage points higher than the simulated result.

Measurement	Miss Rate	Hit Rate
libCacheSim, LRU, 1 GB	62.97%	37.03%
BlueStore actual	79.9%	22.9%
Gap	+16.9 percentage points	-14.1 percentage points

There are three main reasons for this gap.

First, the BlueStore cache is not dedicated only to user data. The libCacheSim simulation assumes an ideal cache where the full 1 GB cache is available for data blocks. However, in the real Ceph system, the BlueStore cache is shared among data blocks, onode metadata, RocksDB key-value entries, allocation metadata, and other internal structures. Therefore, the effective cache capacity available for user data is smaller than 1 GB.

Second, replication increases cache pressure. Since the replication factor was 3, each logical write was stored on three different OSDs. Each OSD has its own independent BlueStore cache. As a result, the workload is distributed across multiple independent caches rather than managed by a single unified cache, as assumed in the simulation. This cache fragmentation can reduce caching efficiency.

Third, libCacheSim uses an idealized cache replacement model. BlueStore's actual cache eviction behavior is more complex than a simple LRU policy. It must manage multiple types of cached data, including user data, metadata, onodes, and RocksDB-related entries. Therefore, data that would remain cached in an ideal simulation may be evicted in the real system to make room for metadata or other internal structures.

For these reasons, the libCacheSim results are useful as a theoretical baseline, but they do not fully reproduce the cache behavior of a real Ceph BlueStore deployment.

7.3 Read Latency Caveat

In this experiment, the average read latency was only 0.056 ms. However, this value should not be interpreted as the true disk-level read performance of Ceph.

The main reason is the effect of the Linux page cache. Although the page cache was dropped before the replay started, write operations during the replay could repopulate the client-side page cache. If later read operations accessed the same or nearby addresses, those reads could be served from the Linux page cache instead of reaching the Ceph OSDs.

This interpretation is supported by the BlueStore performance counters. Although the trace contained 46,974 read requests, only 1,902 random reads reached BlueStore. This suggests that many read requests were likely served by the client-side page cache rather than the Ceph backend.

Therefore, the measured read latency in this experiment is strongly influenced by the Linux page cache and should not be treated as pure Ceph OSD or HDD read latency. To measure Ceph read performance more accurately, a separate read-only workload should be issued against pre-written data after carefully controlling both the Linux page cache and the BlueStore cache.

This is an important limitation of the current experiment, and future experiments should include a more controlled read performance measurement.

8 Limitations and Future Work

The following limitations should be considered when interpreting the results.

- Read latency measurements were dominated by the Linux page cache rather than Ceph OSD performance, as discussed in Section 7.3.
- The replay was single-threaded and does not reflect the concurrent I/O behavior of real cloud workloads.
- Only one trace was used, which limits generalizability to other workload types.
- Each configuration was run only once; averaging multiple runs would improve measurement reliability.

8.1 Future Work

Future work should first measure read performance more accurately. One possible approach is to issue a read-only workload against pre-written data while carefully controlling both the Linux page cache and the BlueStore cache. This would make it possible to measure the actual read latency of the Ceph OSDs and HDDs more accurately.

Another direction is to compare different replication factors. This experiment used a replication factor of 3, but testing a replication factor of 2 or other redundancy configurations would help evaluate how write latency and durability change under different settings. This would make the latency-durability trade-off clearer.

It would also be useful to test different BlueStore cache sizes. In this experiment, the BlueStore cache size was 1 GB per OSD. By increasing or decreasing the cache size, future experiments could examine how the gap between the libCacheSim simulation results and the actual BlueStore cache performance changes.

In addition, multi-threaded replay should be introduced to create a workload that is closer to real cloud storage environments. By issuing I/O requests from multiple threads or clients, future experiments could measure how much throughput Ceph can achieve under concurrent workloads.

Finally, future work should evaluate Ceph under a degraded cluster state. For example, replaying the trace while one OSD is down would help measure how write latency, read performance, and availability change during failures. This would be important for understanding the fault tolerance behavior of distributed storage systems such as Ceph.

9 Conclusion

This study evaluated the performance of Ceph by deploying a three-node Ceph cluster on CloudLab and replaying a real-world block I/O trace named `cloudPhysicsIO.vscsi`. The experiment included workload analysis, cache simulation with libCacheSim, real I/O replay against Ceph RBD, and cache behavior measurement using BlueStore performance counters.

The workload analysis showed that the trace was write-dominant, with write requests accounting for 58.7% of all requests. The most frequent I/O size was 64 KB, and the workload showed bursty behavior. These characteristics suggest that the trace represents a more realistic storage workload than a simple synthetic benchmark.

The libCacheSim simulation predicted a miss ratio of 62.97% for LRU with a 1 GB cache. However, the actual BlueStore performance counters showed a cache miss rate of 79.9%. This gap can be explained by several factors. First, the BlueStore cache is not dedicated only to user data, but is also used for metadata and RocksDB-related information. Second, the replication factor of 3 distributes data across multiple independent OSD caches. Third, BlueStore's actual cache eviction behavior is more complex than the idealized LRU model used in the simulation.

The real I/O replay showed that the average write latency was 24.543 ms, and the p99 write latency was 95.260 ms. These values are reasonable for a Ceph cluster using HDDs with 3-way replication. They indicate that Ceph write performance is affected by disk latency, replication, and BlueStore commit processing. In contrast, the measured read latency was very low, but

this result should be interpreted carefully because it was likely influenced by the Linux page cache rather than pure Ceph OSD read performance.

Overall, the results show that there can be a substantial gap between cache behavior predicted by simulation and performance observed in a real distributed storage system. In a system such as Ceph, performance is affected by replication, metadata management, backend cache design, and operating-system-level caching. Therefore, understanding distributed storage performance requires not only simulation, but also measurement and analysis on a real cluster.

References

[1] S. A. Weil, S. A. Brandt, E. L. Miller, D. D. E. Long, and C. Maltzahn, "Ceph: A Scalable, High-Performance Distributed File System," in *Proceedings of the 7th USENIX symposium on operating systems design and implementation (OSDI '06)*, Seattle, WA, USA, Nov. 2006. Available: https://www.usenix.org/conference/osdi-06/ceph-scalable-high-performance-distributed-file-system

[2] Ceph Documentation, "Ceph block device." Accessed: May 30, 2026. [Online]. Available: https://docs.ceph.com/en/latest/rbd/

[3] Ceph Documentation, "BlueStore Configuration Reference." Accessed: May 30, 2026. [Online]. Available: https://docs.ceph.com/en/quincy/rados/configuration/bluestore-config-ref/

[4] Ceph Documentation, "Perf counters." Accessed: May 30, 2026. [Online]. Available: https://docs.ceph.com/en/latest/dev/perf_counters/

[5] libCacheSim Developers, "libCacheSim: a high performance cache simulator." GitHub repository. Accessed: May 30, 2026. [Online]. Available: https://github.com/cacheMon/libCacheSim

[6] The CloudLab Team, "The CloudLab manual." Accessed: May 30, 2026. [Online]. Available: https://docs.cloudlab.us/

[7] M. Kerrisk, "open(2) — Linux manual page." man7.org. Accessed: May 30, 2026. [Online]. Available: https://man7.org/linux/man-pages/man2/open.2.html